

Correction Partie A

A- REM Création des types

1. CREATE TYPE adresse_type AS OBJECT
(numero integer,
rue varchar(30),
ville varchar(20))

2. CREATE TYPE departement_type AS OBJECT
(numDept smallint,
nomDept varchar(20),
lieu varchar(50))

3. CREATE TYPE employe_type AS OBJECT
(matricule smallint,
nom varchar(30),
adresse adresse_type,
salaire decimal(8,2),
superieur ref employe_type,
departement ref departement_type)

4. REM Création des tables

create table departement OF departement_type
(constraint pk_departement primary key (numDept))

create table employe OF employe_type
(constraint pk_employe primary key (matricule))

5. describe departement_type

desc employe_type
desc departement
desc employe

6. select type_name from user_types

Ajouter des données dans les nouvelles tables

1. Ajoutez des données dans les nouvelles tables (au moins 2 départements et 3 employés). Attention, la version actuelle (fin 2010) de SQL Developer n'affiche pas toujours correctement les références. Pour vérifier, vous pouvez lancer un select qui renvoie les valeurs du type référencé, par exemple `departement.numDept`.
2. Faites afficher pour chaque employé : le matricule, la ville où il habite, le nom de son supérieur, la ville où il travaille (celle de son département).
3. Faites afficher le nom des employés du département 10 (ou d'un autre numéro).

Solution

REM 1. Insertion des données

```
insert into departement(numDept, nomDept, lieu)
values(10, 'Direction', 'Nice')
```

REM Une autre façon d'ajouter une ligne en la considérant comme un objet :

```
insert into departement values(
    departement_type(20, 'Comptabilité', 'Marseille'))
```

```
insert into employe (matricule, nom, adresse, salaire, superieur, departement)
values(125, 'Dupond', adresse_type(15, 'rue Victor Hugo', 'Nice'),
    12500, null, null)
```

```
insert into employe (matricule, nom, salaire, superieur, departement)
values(200, 'Leroy', 25000, null, null)
```

```
insert into employe (matricule, nom, salaire, superieur, departement)
select 210, 'Ravier', 25000, ref(e), ref(d)
from employe e, departement d
where e.matricule = 125 and d.numDept = 10
```

```
insert into employe (matricule, nom, salaire, superieur, departement)
select 300, 'Toto', 20000, ref(e), ref(d) from employe e, departement d
where e.matricule = 210 and d.numDept = 10;
```

REM autre solution possible dans les dernières versions d'Oracle

REM sans doute pas portable. Ne pas oublier les parenthèses autour des selects,

```
insert into employe (matricule, nom, salaire, superieur, departement)
values (300, 'Toto', 20000,
       (select ref(e) from employe e where e.matricule = 210),
       (select ref(d) from departement d where d.numDept = 10))
```

REM 2. Afficher des informations sur les employés

```
select matricule, nom, e.adresse.ville, e.superieur.nom, e.departement.lieu
from employe e;
```

REM 3. Employés du département 10

```
select nom, e.departement.numdept
from employe e
where e.departement.numdept = 10;
```

Modifier les données

1. Changez le supérieur et le département d'un des employés.
2. Mettez un des employés comme supérieur de tous les employés du département numéro 10.
3. Essayez de mettre un employé dans un département qui a un numéro qui n'existe pas.

Correction

```
1. update employe
   set departement =
     (select REF(d)
      from departement d
      where numDept = 10)
  where matricule = 125
```

```
update employe
   set superieur =
     (select REF(e)
      from employe e
      where matricule = 200)
  where matricule = 125
```

En une fois :

```
update employe
  set superieur =
    (select REF(e)
     from employe e
     where matricule = 200),
  departement =
    (select REF(d)
     from departement d
     where numDept = 10)
where matricule = 125
```

2.

```
update employe
  set superieur = (select ref(e) from employe e where matricule = 200)
where departement = (select ref(d) from departement d where numdept = 10)
```

3. update employe

```
  set departement =
    (select REF(d)
     from departement d
     where numDept = 85)
where matricule = 200
```

Si on n'a pas mis de contrainte d'intégrité de référence, pas de message d'erreur mais aucun département n'est associé à l'employé, même si on ajoute ensuite un département de numéro 85. C'est normal car on a seulement dit que departement était de type departement_type, sans dire que departement référençait une ligne de la table departement.

Un moyen simple d'empêcher ça est d'ajouter une contrainte "not null" sur la colonne departement. En effet, le fait de donner un mauvais numéro pour numDept renvoie null pour la colonne departement :

```
alter table employe
  add (constraint nonnul_departement check(departement is not null));
```

Remarque : pour que cette contrainte soit acceptée il faut évidemment que tous les employés déjà entrés aient un département.

Pour que l'intégrité de référence marche aussi dans l'autre sens (des départements vers les employés) et qu'il ne soit pas possible de supprimer un département s'il contient des employés, ça ne suffit pas. Il faut en plus ajouter une contrainte de référence à la colonne departement de la table employe :

`ALTER TABLE employe ADD(SCOPE FOR (departement) IS departement);`

Mais ça ne marche que si la table employe est vide ! Il faut donc vider la table des employés et réinsérer les employés ensuite (après les avoir sauveés dans une autre table par exemple).

Ca ne suffit pas pour éviter les références "pendantes". Une contrainte "references" ajoute un contrôle supplémentaire :

```
alter table employe3
add constraint ref_employe3_departement
foreign key(departement) references departement;
```

Ensuite on ne pourra plus supprimer un département qui contient des employés. Essayez par exemple

```
delete from departement
where numdept = 10;
```

Array

1. Créez un nouveau type `personne_type` avec un nom et un tableau de prénoms (au plus 4 prénoms).
2. Créez la table `personne` correspondante (n'oubliez pas les contraintes d'intégrité).
3. Faites afficher une description de `personne` avec la commande `describe`.
4. Ajoutez des lignes dans la table `personne`.
5. Affichez les noms et tous les prénoms que vous avez entrés.

Correction

```
create type listePrenoms as VARRAY(4) of varchar(20);
CREATE TYPE personne_type AS OBJECT
(nom varchar(30),
prenoms listePrenoms);
create table personne OF personne_type (primary key(nom));
(une clé non significative serait meilleure).
describe personne
insert into personne
values('Machin', listePrenoms('Bernard', 'Alain'));
select select nom, prenoms from personne;
```

Vous voulez le 1er prénom de chacune des personnes ? On peut y arriver mais je ne connais aucune solution qui fonctionne sur tous les SGBD. Si vous y tenez vraiment, voici une solution avec Oracle (inutile de rentrer dans les détails : cette solution vous est juste donnée à titre indicatif) :

```
select nom, (select * from table(personne.prenoms) where rownum = 1)
from personne
```

Héritage

1. Créez un type `travailleur_type` qui hérite de `personne_type`. Ce nouveau type a seulement un attribut salaire en plus.
2. Créez une table `travailleur` (n'oubliez pas les contraintes d'intégrité).
3. Ajoutez quelques travailleurs dans cette table.
4. Faites afficher des informations sur les travailleurs.

Correction

```
create type travailleur_type under personne_type
(salaire numeric(8,2));
```

Ca ne marche pas car un type est final par défaut. Il faut d'abord rendre `personne_type` non final :

```
alter type personne_type not final cascade;
```

Ensuite on peut lancer la première commande.

Création de la table des travailleurs :

```
create table travailleur of travailleur_type (primary key(nom));
```

Insertion des travailleurs dans la table :

```
insert into travailleur (nom, prenom, salaire)
values('Machin', listePrenoms('Bernard', 'Alain'), 5000);
```

Affichage des informations :

```
select nom, salaire from travailleur;
```